In [ ]:
```python
import marimo as mo
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

from torch.utils.data import DataLoader, Subset
```

In [ ]:
```python
BATCH_SIZE = 32
EPOCHS = 3

NUM_STEPS = 100
DT = 0.025  # мс; всего 2.5 мс модельного времени

INPUT_SIZE = 28 * 28
HIDDEN_SIZE = 128
OUTPUT_SIZE = 10

# HH: классические параметры мембраны
C_M = 1.0
E_NA = 50.0
E_K = -77.0
E_L = -54.387

G_NA = 120.0
G_K = 36.0
G_L = 0.3

# Ток, создаваемый яркостью пикселя
INPUT_CURRENT = 20.0

# Масштаб тока от обучаемых синапсов
SYNAPSE_SCALE = 50.0

# Размеры выборок для первого эксперимента
MAX_TRAIN_SAMPLES = 6000
MAX_TEST_SAMPLES = 2000

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

mo.md(
    f"""
    # HH-SNN для MNIST

    - Устройство: **{device}**
    - Входных HH-нейронов: **{INPUT_SIZE}**
    - Скрытых HH-нейронов: **{HIDDEN_SIZE}**
    - Выходных HH-нейронов: **{OUTPUT_SIZE}**
    - Временных шагов: **{NUM_STEPS}**
    - Шаг интегрирования: **{DT} мс**
    - Время моделирования: **{NUM_STEPS * DT:.2f} мс**
```

```
    - Обучающих примеров: **{MAX_TRAIN_SAMPLES}**
    - Тестовых примеров: **{MAX_TEST_SAMPLES}**
    """
)
```

HH-SNN для MNIST

- Устройство: **cuda**
- Входных HH-нейронов: **784**
- Скрытых HH-нейронов: **128**
- Выходных HH-нейронов: **10**
- Временных шагов: **100**
- Шаг интегрирования: **0.025 мс**
- Время моделирования: **2.50 мс**
- Обучающих примеров: **6000**
- Тестовых примеров: **2000**

In [ ]:
```
transform = transforms.ToTensor()

full_train_dataset = torchvision.datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

full_test_dataset = torchvision.datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

# Случайные подвыборки для ускорения первого запуска
generator = torch.Generator().manual_seed(42)

train_indices = torch.randperm(
    len(full_train_dataset),
    generator=generator
)[:MAX_TRAIN_SAMPLES].tolist()

test_indices = torch.randperm(
    len(full_test_dataset),
    generator=generator
)[:MAX_TEST_SAMPLES].tolist()

train_dataset = Subset(
    full_train_dataset, train_indices
)
```

```
test_dataset = Subset(
    full_test_dataset, test_indices
)

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=0
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0
)

mo.md(
    f"""
    ## MNIST загружен

    - Обучение: {len(train_dataset)} изображений
    - Тестирование: {len(test_dataset)} изображений
    """
)
```

MNIST загружен

- Обучение: 6000 изображений
- Тестирование: 2000 изображений

In [ ]:
```
class SurrogateSpike(torch.autograd.Function):
    """
    Прямой проход: настоящий бинарный спайк.
    Обратный проход: приближённый градиент.
    """

    @staticmethod
    def forward(ctx, voltage):
        ctx.save_for_backward(voltage)
        return (voltage > 0.0).to(voltage.dtype)

    @staticmethod
    def backward(ctx, grad_output):
        (voltage,) = ctx.saved_tensors

        # Сглаженный градиент пороговой функции.
        # Масштаб подобран для потенциала в мВ.
        slope = 0.1

        grad = 1.0 / (
            1.0 + slope * voltage.abs()
```

```
        ).square()

        return grad_output * grad

def spike_function(voltage):
    return SurrogateSpike.apply(voltage)

class HHLayer(nn.Module):
    """
    Популяция независимых HH-нейронов.
    Синаптические токи поступают извне.
    """

    def __init__(self, size):
        super().__init__()
        self.size = size

    def initial_state(self, reference):
        shape = reference.shape

        V = torch.full(
            shape, -65.0,
            device=reference.device,
            dtype=reference.dtype
        )

        m = torch.full_like(V, 0.0529)
        h = torch.full_like(V, 0.5961)
        n = torch.full_like(V, 0.3177)

        return V, m, h, n

    def step(self, current, state):
        V, m, h, n = state

        # Скорости HH.
        # Защита от сингулярностей при V=-40 и V=-55 мВ.
        xm = V + 40.0
        denom_m = -torch.expm1(-xm / 10.0)
        denom_m = torch.where(
            xm.abs() < 1e-6,
            torch.ones_like(denom_m),
            denom_m
        )

        am_regular = 0.1 * xm / denom_m
        alpha_m = torch.where(
            xm.abs() < 1e-6,
            torch.ones_like(V),
            am_regular
        )

        beta_m = 4.0 * torch.exp(
            -(V + 65.0) / 18.0
        )
```

```
        alpha_h = 0.07 * torch.exp(
            -(V + 65.0) / 20.0
        )

        beta_h = 1.0 / (
            1.0 + torch.exp(-(V + 35.0) / 10.0)
        )

        xn = V + 55.0
        denom_n = -torch.expm1(-xn / 10.0)
        denom_n = torch.where(
            xn.abs() < 1e-6,
            torch.ones_like(denom_n),
            denom_n
        )

        an_regular = 0.01 * xn / denom_n
        alpha_n = torch.where(
            xn.abs() < 1e-6,
            torch.full_like(V, 0.1),
            an_regular
        )

        beta_n = 0.125 * torch.exp(
            -(V + 65.0) / 80.0
        )

        # Ионные токи.
        I_Na = (
            G_NA * m.pow(3) * h * (V - E_NA)
        )

        I_K = (
            G_K * n.pow(4) * (V - E_K)
        )

        I_L = G_L * (V - E_L)

        # Уравнения динамики.
        dV = (
            current - I_Na - I_K - I_L
        ) / C_M

        dm = (
            alpha_m * (1.0 - m)
            - beta_m * m
        )

        dh = (
            alpha_h * (1.0 - h)
            - beta_h * h
        )

        dn = (
            alpha_n * (1.0 - n)
            - beta_n * n
```

```
        )

        # Интегрирование явным методом Эйлера.
        V_next = V + DT * dV

        m_next = (m + DT * dm).clamp(0.0, 1.0)
        h_next = (h + DT * dh).clamp(0.0, 1.0)
        n_next = (n + DT * dn).clamp(0.0, 1.0)

        # Спайк только регистрируется.
        # Потенциал мембраны вручную не сбрасывается.
        spikes = spike_function(V_next)

        next_state = (
            V_next, m_next, h_next, n_next
        )

        return spikes, next_state

class FullHHNet(nn.Module):
    def __init__(self):
        super().__init__()

        self.num_steps = NUM_STEPS

        # Обучаемые синаптические связи.
        self.fc1 = nn.Linear(
            INPUT_SIZE, HIDDEN_SIZE
        )

        self.fc2 = nn.Linear(
            HIDDEN_SIZE, OUTPUT_SIZE
        )

        # Инициализация весов.
        nn.init.xavier_uniform_(self.fc1.weight)
        nn.init.zeros_(self.fc1.bias)

        nn.init.xavier_uniform_(self.fc2.weight)
        nn.init.zeros_(self.fc2.bias)

        # HH-нейроны на всех уровнях.
        self.hh_in = HHLayer(INPUT_SIZE)
        self.hh_hidden = HHLayer(HIDDEN_SIZE)
        self.hh_out = HHLayer(OUTPUT_SIZE)

    def forward(self, images):
        batch_size = images.shape[0]

        x = images.reshape(batch_size, -1)

        # Каждая популяция имеет собственные состояния.
        state_in = self.hh_in.initial_state(x)

        hidden_reference = x.new_zeros(
            batch_size, HIDDEN_SIZE
```

```
        )

        output_reference = x.new_zeros(
            batch_size, OUTPUT_SIZE
        )

        state_hidden = self.hh_hidden.initial_state(
            hidden_reference
        )

        state_out = self.hh_out.initial_state(
            output_reference
        )

        output_spikes = []

        for _ in range(self.num_steps):
            # Вход: яркость пикселя задаёт ток.
            input_current = x * INPUT_CURRENT

            spk_in, state_in = self.hh_in.step(
                input_current, state_in
            )

            # Скрытый слой получает взвешенные спайки.
            hidden_current = (
                self.fc1(spk_in) * SYNAPSE_SCALE
            )

            spk_hidden, state_hidden = (
                self.hh_hidden.step(
                    hidden_current, state_hidden
                )
            )

            # Выходной слой.
            output_current = (
                self.fc2(spk_hidden) * SYNAPSE_SCALE
            )

            spk_out, state_out = self.hh_out.step(
                output_current, state_out
            )

            output_spikes.append(spk_out)

        # [время, batch, 10]
        return torch.stack(output_spikes, dim=0)

model = FullHHNet()
```

In [ ]:

```
def run_training():
    model.to(device)

    optimizer = torch.optim.Adam(
        model.parameters(),
```

```
    lr=1e-4
)

criterion = nn.CrossEntropyLoss()

losses = []
train_accuracies = []
test_accuracies = []

def evaluate(loader):
    model.eval()

    correct = 0
    total = 0

    with torch.no_grad():
        for batch_images, batch_labels in loader:
            batch_images = batch_images.to(device)
            batch_labels = batch_labels.to(device)

            output_spikes = model(batch_images)
            counts = output_spikes.sum(dim=0)

            predictions = counts.argmax(dim=1)

            correct += (
                predictions == batch_labels
            ).sum().item()

            total += batch_labels.size(0)

    return 100.0 * correct / max(total, 1)

for epoch in range(EPOCHS):
    model.train()

    total_loss = 0.0
    correct = 0
    total = 0

    for batch_images, batch_labels in train_loader:
        batch_images = batch_images.to(device)
        batch_labels = batch_labels.to(device)

        optimizer.zero_grad(set_to_none=True)

        output_spikes = model(batch_images)
        counts = output_spikes.sum(dim=0)

        loss = criterion(counts, batch_labels)
        loss.backward()

        # Защита от слишком больших градиентов.
        torch.nn.utils.clip_grad_norm_(
            model.parameters(), max_norm=1.0
        )
```

```
            optimizer.step()

            total_loss += loss.item()

            predictions = counts.detach().argmax(dim=1)

            correct += (
                predictions == batch_labels
            ).sum().item()

            total += batch_labels.size(0)

        mean_loss = total_loss / max(len(train_loader), 1)
        train_acc = 100.0 * correct / max(total, 1)
        test_acc = evaluate(test_loader)

        losses.append(mean_loss)
        train_accuracies.append(train_acc)
        test_accuracies.append(test_acc)

        print(
            f"Epoch {epoch + 1}/{EPOCHS} | "
            f"loss={mean_loss:.4f} | "
            f"train={train_acc:.2f}% | "
            f"test={test_acc:.2f}%"
        )

    return losses, train_accuracies, test_accuracies

train_losses, train_accuracies, test_accuracies = (
    run_training()
)

mo.md("## Обучение завершено")
```

```
Epoch 1/3 | loss=2.1875 | train=13.40% | test=24.35%
Epoch 2/3 | loss=1.5648 | train=40.37% | test=58.45%
Epoch 3/3 | loss=0.8891 | train=69.53% | test=78.20%
```

Обучение завершено

In [ ]:

```
def plot_curves():
    fig, axes = plt.subplots(
        1, 2, figsize=(11, 4)
    )

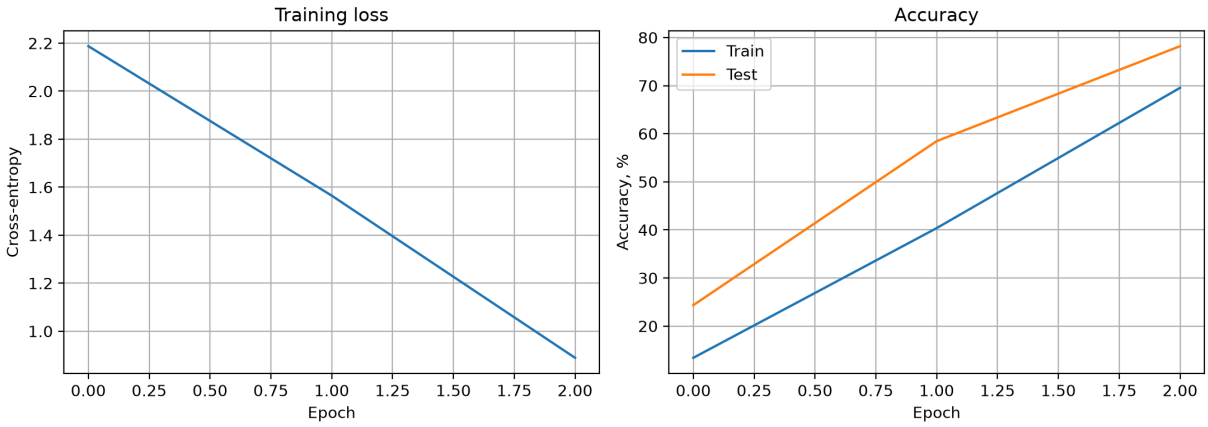
    axes[0].plot(train_losses)
    axes[0].set_title("Training loss")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Cross-entropy")
    axes[0].grid(True)

    axes[1].plot(
```

```
        train_accuracies, label="Train"
    )
    axes[1].plot(
        test_accuracies, label="Test"
    )
    axes[1].set_title("Accuracy")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Accuracy, %")
    axes[1].legend()
    axes[1].grid(True)

    fig.tight_layout()
    plt.show()

plot_curves()
```



In [ ]:

```
def show_predictions():
    model.eval()

    count = 10

    sample_images = torch.stack([
        test_dataset[i][0] for i in range(count)
    ]).to(device)

    true_labels = torch.tensor([
        test_dataset[i][1] for i in range(count)
    ])

    with torch.no_grad():
        output_spikes = model(sample_images)
        spike_counts = output_spikes.sum(dim=0)
        predicted_labels = (
            spike_counts.argmax(dim=1).cpu()
        )

    fig, axes = plt.subplots(
        2, 5, figsize=(12, 5)
    )

    for i, ax in enumerate(axes.flat):
        ax.imshow(
```

```
            sample_images[i, 0].cpu().numpy(),
            cmap="gray"
        )

        ax.set_title(
            f"Предсказание: {predicted_labels[i].item()}\n"
            f"Истина: {true_labels[i].item()}"
        )

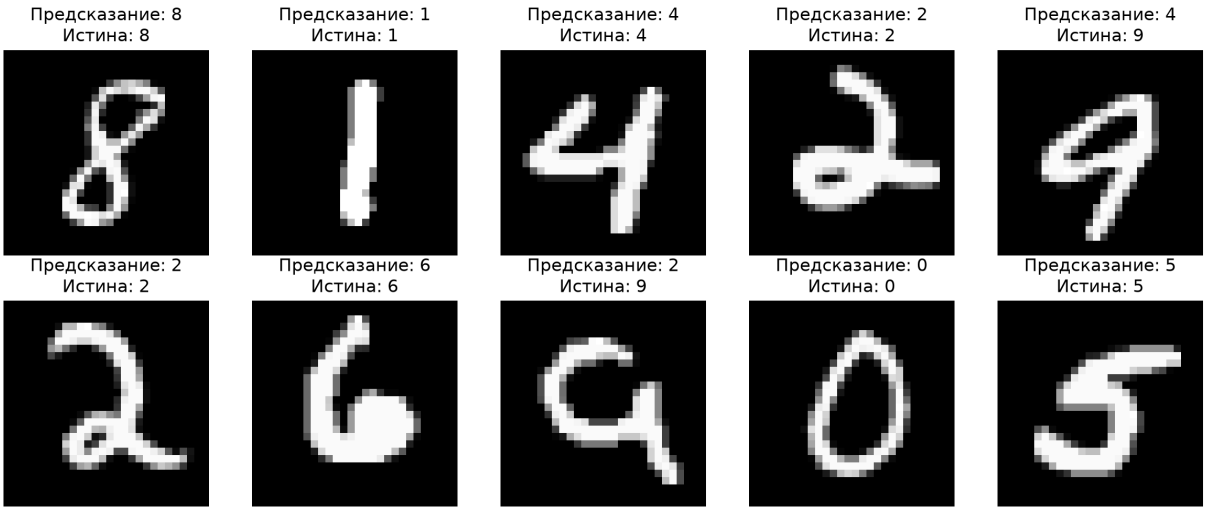
        ax.axis("off")

    fig.tight_layout()
    plt.show()

show_predictions()

mo.md(
    """
    ## Интерпретация

    Каждый выходной НН-нейрон соответствует одной цифре.
    Предсказание определяется по количеству спайков
    за весь интервал моделирования.
    """
)
```



Интерпретация

Каждый выходной НН-нейрон соответствует одной цифре. Предсказание определяется по количеству спайков за весь интервал моделирования.